Utica University | Department of Computer Science | Fall 2026

CSC 618: Information Retrieval | Week 10 Assignment

Submission 4: Part C, Experiment on a Real Online IR Dataset

November 2026

1. Summary

I ran TF-IDF, BM25 and a Dirichlet Language Model on the Cranfield test collection using Apache Lucene. With default parameters, BM25 (MAP 0.2952) and TF-IDF (MAP 0.2931) were statistically indistinguishable, and both were significantly better than the Language Model (MAP 0.2341). Every value in this report comes from an actual run.

2. Dataset description and source

Table 1: Cranfield collection.

Documents	1400 aeronautical engineering abstracts; 143285 tokens after analysis, 5172 unique terms, mean 102.3 terms per document
Queries	225 natural-language questions, mean 18.0 words
Qrels	1837 rows, 1612 of them positive; mean 7.16 relevant documents per query, from 1 to 39
Why chosen	It is public, has the components the assignment requires, and is a historic benchmark [5]. TREC document sets usually require a license [6].
Source used	TREC-format copy, commit 1208e6e: `https://github.com/oussbenk/cranfield-trec-dataset`
Original	`http://ir.dcs.gla.ac.uk/resources/test_collections/cran/`

I could not reach the original site from my working environment, so I used the TREC-format copy and checked it against the `ir_datasets` metadata for Cranfield [7]. The document, query and qrel counts match (1400, 225, 1837), and the 225 rows labeled 0 match the metadata's 225 "no interest" (-1) rows. The metadata lists grades 1 to 4 for the other rows, and this copy has flattened them to 1, apart from one row labeled 3 that I treated as relevant. Documents 471 and 995 have no abstract, so I indexed their titles.

3. Experimental setup and methodology

Table 2: Setup, the same for all models.

Engine	Apache Lucene 10.5.0, Java 21 (jar from Anserini 2.3.0 [8], distributed with Pyserini 2.4.0)
Analysis	`EnglishAnalyzer` on documents and queries (default stop words, Porter stemmer)
Indexed text	Abstract field, which already begins with the title in the source
Query	Topic text analyzed, then all terms combined with OR (`BooleanQuery`, SHOULD)
Depth	Top 1000; a mean of 891 documents returned, since only documents with a query term match
Saved results	TREC-format run files (`results/partc_*.run`)
Evaluation	Part B functions; set metrics on the full list and at rank 10; AP over the list; nDCG with binary gain
Testing	Paired two-sided randomization test (100,000 sign flips, seed 618) and Wilcoxon signed-rank test [9]

Table 3: Models. f is a query term's frequency in document d and $|d|$ the document length.

Model	Scoring idea	Parameters
TF-IDF (`ClassicSimilarity`)	$\sqrt{f} \cdot \text{idf}^2 \cdot |d|^{-1/2}$, summed over query terms	defaults
BM25 (`BM25Similarity`) [10]	idf times a saturating function of f, normalized by $|d|$ relative to the mean	k_1=1.2, b=0.75
LM-Dirichlet (`LMDirichletSimilarity`) [11]	Term probability in d, smoothed with the collection model	μ=2000

4. Results

Table 4: Mean metrics over 225 queries. The best value in each row is in bold; ties are not marked.

Metric	TF-IDF	BM25	LM-Dirichlet
MAP	0.2931	**0.2952**	0.2341
MRR	**0.5327**	0.5204	0.4557
P@3	0.3452	**0.3570**	0.2726
P@10	0.2244	**0.2284**	0.1840
Recall@10	0.3806	**0.3909**	0.3248
F1@10	0.2544	**0.2600**	0.2119
nDCG@5	**0.3663**	0.3651	0.2858
nDCG@10	0.3690	**0.3741**	0.3041
Precision (full list)	0.0081	0.0081	0.0081
Recall (full list)	0.9513	**0.9523**	0.9515
F1 (full list)	0.0160	0.0160	0.0160

Table 5: Paired tests over 225 queries. W/L/T counts queries where the first model scored higher, lower or equal.

Comparison	Metric	Mean diff.	W/L/T	Randomization p	Wilcoxon p
BM25 vs TF-IDF	AP	+0.0021	113/101/11	0.7474	0.7537
BM25 vs TF-IDF	nDCG@10	+0.0050	92/79/54	0.5214	0.4959
BM25 vs TF-IDF	RR	−0.0123	62/64/99	0.4803	0.7383
BM25 vs TF-IDF	P@10	+0.0040	40/29/156	0.4474	0.4442
BM25 vs LM-Dirichlet	AP	+0.0611	169/49/7	$< 10^{-5}$	$< 10^{-5}$
BM25 vs LM-Dirichlet	nDCG@10	+0.0700	133/50/42	$< 10^{-5}$	$< 10^{-5}$
BM25 vs LM-Dirichlet	RR	+0.0647	96/41/88	0.00049	0.00009
BM25 vs LM-Dirichlet	P@10	+0.0444	87/22/116	$< 10^{-5}$	$< 10^{-5}$
TF-IDF vs LM-Dirichlet	AP	+0.0589	157/62/6	$< 10^{-5}$	$< 10^{-5}$
TF-IDF vs LM-Dirichlet	nDCG@10	+0.0649	123/68/34	$< 10^{-5}$	$< 10^{-5}$
TF-IDF vs LM-Dirichlet	RR	+0.0770	94/56/75	0.0019	0.0023
TF-IDF vs LM-Dirichlet	P@10	+0.0404	90/31/104	$< 10^{-5}$	$< 10^{-5}$

With a Bonferroni threshold of $0.05/12 = 0.0042$, all eight comparisons involving the Language Model stay significant and none of the BM25 vs TF-IDF comparisons is.

Table 6: Query-level behavior. All three models tied for best AP on 10 queries.

	TF-IDF	BM25	LM-Dirichlet
Queries where the model alone has the best AP	99	75	41
Queries with no relevant document in the top 10	34	32	50

Mean top-10 overlap was 0.762 for BM25 and TF-IDF, 0.596 for BM25 and the Language Model, and 0.490 for TF-IDF and the Language Model.

Table 7: Five queries with the largest AP spread across models.

ID	Query	Rel.	TF-IDF	BM25	LM
52	what is the available information pertaining to the effect of slight rarefaction on boundary layer flows (the ?slip? effect) .	4	0.812	0.533	0.118
165	are the stable profiles of a compressible boundary layer induced by a moving wave known .	2	0.833	0.583	0.161
81	what are wind-tunnel corrections for a two-dimensional aerofoil mounted off-centre in a tunnel .	2	0.417	0.833	0.212
143	what general solutions for the stresses in pressurized shells of revolution are available .	2	0.417	0.625	0.111
119	what is the effect of initial axisymmetric deviations from circularity on the non linear (large-deflection) load-deflection response of cylinders under hydrostatic pressure .	1	1.000	1.000	0.500

Table 8: Parameter sensitivity. These settings were picked after viewing test queries, with no separate tuning set, so treat them as exploratory.

Configuration	MAP	MRR	nDCG@10
BM25, b=0.25	0.2728	0.4993	0.3492
BM25, b=0.5	0.2850	0.5100	0.3609
BM25, b=0.75 (default)	0.2952	0.5204	0.3741
BM25, b=1.0	0.2938	0.5224	0.3716
LM-Dirichlet, μ=50	0.2757	0.5173	0.3548
LM-Dirichlet, μ=100	0.2724	0.5083	0.3489
LM-Dirichlet, μ=200	0.2666	0.4908	0.3410
LM-Dirichlet, μ=500	0.2545	0.4835	0.3304
LM-Dirichlet, μ=1000	0.2449	0.4709	0.3176
LM-Dirichlet, μ=2000 (default)	0.2341	0.4557	0.3041
LM-JM, λ=0.1	0.2591	0.5079	0.3372
LM-JM, λ=0.3	0.2719	0.5113	0.3534
LM-JM, λ=0.7	0.2758	0.5074	0.3527

5. Why performance differs

BM25 and TF-IDF behave alike here. Their top-10 lists overlap by about three quarters, and both use IDF and length normalization. My guess is that Cranfield's short abstracts give BM25's term-frequency saturation few chances to matter, but I did not test this directly.

The Language Model is the outlier. MAP improves steadily as μ falls from 2000 to 50 (0.2341 to 0.2757), which fits over-smoothing, since μ=2000 is about 20 times the mean document length. Tuning does not close the gap to BM25, and Jelinek-Mercer smoothing tops out at a similar value (0.2758). Lucene's documentation says this class returns 0 where the formula would give a negative score, and only documents containing a query term are scored. Those choices may also limit it, but I did not isolate their effect.

The means hide differences between queries. On query 81, BM25's AP (0.833) is double TF-IDF's (0.417), and on query 52 TF-IDF leads BM25 by 0.28.

6. Verification

My metrics matched the independent `ranx` library [12] to six decimal places on all eight metrics I compared (three are shown below). A hand check agrees too: for query 81 under BM25, relevant documents 799 and 672 are at ranks 1 and 3, so AP $= (1/1 + 2/3)/2 = 0.8333$ and RR $= 1$, the same as the program.

Table 9: My implementation vs ranx.

Model	MAP (mine)	MAP (ranx)	MRR (mine)	MRR (ranx)	nDCG@10 (mine)	nDCG@10 (ranx)
TF-IDF	0.293067	0.293067	0.532660	0.532660	0.369045	0.369045
BM25	0.295186	0.295186	0.520377	0.520377	0.374072	0.374072
LM-Dirichlet	0.234132	0.234132	0.455700	0.455700	0.304098	0.304098

7. Limitations

- The collection is small, old and domain-specific, so these rankings need not hold for other search tasks.
- The qrels copy is binary, so nDCG here does not measure graded relevance. That would need the original graded file.
- Unjudged documents count as non-relevant.
- The main comparison uses default parameters, and the sensitivity table had no separate tuning set.
- Lucene's versions differ from textbook forms, and the analyzer leaves question words such as "what" in queries.
- There was no user study, and I did not compare speed or index size.

References

[1] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge Univ. Press, 2008, Ch. 8.

[2] Apache Lucene, https://lucene.apache.org/.

[3] Cranfield, TREC-format copy, https://github.com/oussbenk/cranfield-trec-dataset.

[4] Cranfield, original files, http://ir.dcs.gla.ac.uk/resources/test_collections/cran/.

[5] C. Cleverdon, "The Cranfield tests on index language devices," *Aslib Proc.*, 19(6), 1967.

[6] NIST TREC FAQ, https://trec.nist.gov/faq.html.

[7] S. MacAvaney et al., "Simplified data wrangling with ir_datasets," SIGIR 2021.

[8] J. Lin et al., "Pyserini: A Python toolkit for reproducible information retrieval research with sparse and dense representations," SIGIR 2021.

[9] M. Smucker, J. Allan, and B. Carterette, "A comparison of statistical significance tests for information retrieval evaluation," CIKM 2007.

[10] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, 3(4), 2009.

[11] C. Zhai and J. Lafferty, "A study of smoothing methods for language models applied to information retrieval," *ACM TOIS*, 22(2), 2004.

[12] E. Bassani, "ranx: A blazing-fast Python library for ranking evaluation and comparison," ECIR 2022.

Appendix A: Code

The folder part_b_c in https://github.com/Atmosu/csc618-ir-evaluation-rocchio-cranfield holds this code, the main run files and the result tables.

prepare_cranfield.py

```
"""Convert the TREC-XML copy of Cranfield to flat files and validate it.
Query IDs are renumbered 1..225 by file position: the raw <num> values are
non-consecutive (1, 2, 4, 8, ...) while the qrels number queries consecutively.
Usage: prepare_cranfield.py <source_dir> <output_dir>"""
import re, json, sys, collections
root, out = sys.argv[1], sys.argv[2]
clean = lambda s: re.sub(r"\s+", " ", s).strip()
raw = open(f"{root}/cran.all.1400.xml", encoding="utf-8").read()
docs = []
for d in re.findall(r"<doc>(.*?)</doc>", raw, flags=re.S):
    g = lambda t: clean((re.search(fr"<{t}>(.*?)</{t}>", d, flags=re.S) or [None, ""])[1])
    docs.append((g("docno"), g("title"), g("text")))
raw = open(f"{root}/cran.qry.xml", encoding="utf-8").read()
qs = [(int(n), clean(t)) for n, t in re.findall(r"<num>\s*(\d+)\s*</num>\s*<title>(.*?)</title>", raw, flags=re.S)]
rows = [l.split() for l in open(f"{root}/cranqrel.trec.txt")]
with open(f"{out}/docs.tsv", "w") as f:
    for i, t, x in docs: f.write(f"{i}\t{t}\t{x}\n")
with open(f"{out}/queries.tsv", "w") as f:
    for s, (n, t) in enumerate(qs, 1): f.write(f"{s}\t{n}\t{t}\n")
with open(f"{out}/qrels.txt", "w") as f:
    for q, _, d, r in rows: f.write(f"{q} 0 {d} {r}\n")
pos = collections.Counter(q for q, _, d, r in rows if int(r) > 0)
ids = {d[0] for d in docs}
stats = dict(docs=len(docs), unique_doc_ids=len(ids), empty_text_docs=[d[0] for d in docs if not d[2]],
             queries=len(qs), raw_max_query_num=max(n for n, _ in qs), mean_query_words=sum(len(t.split()) for _, t
     in qs) / len(qs),
             qrel_rows=len(rows), qrel_query_ids=len({r[0] for r in rows}), qrel_max_query_id=max(int(r[0]) for r
    in rows),
             qrel_label_counts=dict(collections.Counter(r[3] for r in rows)), qrel_docs_missing=sum(r[2] not in ids
     for r in rows),
             queries_with_relevant=len(pos), positive_rows=sum(pos.values()), relevant_per_query_mean=sum(pos.
    values()) / max(1, len(pos)),
             relevant_per_query_min=min(pos.values()), relevant_per_query_max=max(pos.values()),
             first_raw_ids=[n for n, _ in qs[:6]])
json.dump(stats, open(f"{out}/dataset_stats.json", "w"), indent=1)
print(json.dumps(stats, indent=1))
```

LuceneExperiment.java

```
import java.io.*; import java.nio.file.*; import java.util.*;
import org.apache.lucene.analysis.*; import org.apache.lucene.analysis.en.EnglishAnalyzer;
import org.apache.lucene.analysis.tokenattributes.CharTermAttribute;
import org.apache.lucene.document.*; import org.apache.lucene.index.*; import org.apache.lucene.search.*;
import org.apache.lucene.search.similarities.*; import org.apache.lucene.store.*;

/** Part C: indexes Cranfield once, then runs each retrieval model on the same index and queries.
 *  Usage: LuceneExperiment <dataDir> <resultsDir> */
public class LuceneExperiment {
  static final String F = "contents";
  static final int DEPTH = 1000;
  static Analyzer an = new EnglishAnalyzer();

  static List<String> analyze(String s) throws IOException {
    List<String> o = new ArrayList<>();
    try (TokenStream ts = an.tokenStream(F, s)) {
      CharTermAttribute c = ts.addAttribute(CharTermAttribute.class);
      ts.reset();
      while (ts.incrementToken()) o.add(c.toString());
      ts.end();
    }
    return o;
  }

  static Query orQuery(List<String> toks) {
    BooleanQuery.Builder b = new BooleanQuery.Builder();
    for (String t : toks) b.add(new TermQuery(new Term(F, t)), BooleanClause.Occur.SHOULD);
    return b.build();
  }

  public static void main(String[] a) throws Exception {
    Path data = Paths.get(a[0]), res = Paths.get(a[1]);
    Directory dir = new ByteBuffersDirectory();
    try (IndexWriter w = new IndexWriter(dir, new IndexWriterConfig(an))) {
      for (String l : Files.readAllLines(data.resolve("docs.tsv"))) {
```

```
        String[] p = l.split("\t", -1);
        Document d = new Document();
        d.add(new StringField("id", p[0], Field.Store.YES));
        d.add(new TextField(F, p[2].isEmpty() ? p[1] : p[2], Field.Store.NO)); // text already opens with the title
        w.addDocument(d);
      }
    }
    IndexReader reader = DirectoryReader.open(dir);
    try (PrintWriter w = new PrintWriter(res.resolve("index_stats.txt").toFile())) {
      w.printf(Locale.ROOT, "lucene=%s docs=%d vocab=%d tokens=%d meanLen=%.2f%n", org.apache.lucene.util.Version.
     LATEST, reader.numDocs(),
        MultiTerms.getTerms(reader, F).size(), reader.getSumTotalTermFreq(F), (double) reader.getSumTotalTermFreq(F
     ) / reader.numDocs());
    }
    List<String[]> Q = new ArrayList<>();
    for (String l : Files.readAllLines(data.resolve("queries.tsv"))) Q.add(l.split("\t", -1));

    Map<String, Similarity> sims = new LinkedHashMap<>();
    sims.put("TF-IDF", new ClassicSimilarity());
    sims.put("BM25", new BM25Similarity(1.2f, 0.75f));
    sims.put("LM-Dirichlet", new LMDirichletSimilarity(2000f));
    // Sensitivity variants, reported separately from the default-parameter comparison.
    for (float mu : new float[]{50f, 100f, 200f, 500f, 1000f}) sims.put("LMD-mu" + (int) mu, new
     LMDirichletSimilarity(mu));
    for (float l : new float[]{0.1f, 0.3f, 0.7f}) sims.put("LMJM-l" + l, new LMJelinekMercerSimilarity(l));
    for (float b : new float[]{0.25f, 0.5f, 1.0f}) sims.put("BM25-b" + b, new BM25Similarity(1.2f, b));

    StoredFields sf = reader.storedFields();
    for (var e : sims.entrySet()) {
      IndexSearcher s = new IndexSearcher(reader);
      s.setSimilarity(e.getValue());
      try (PrintWriter w = new PrintWriter(res.resolve("partc_" + e.getKey() + ".run").toFile())) {
        for (String[] q : Q) {
          ScoreDoc[] h = s.search(orQuery(analyze(q[2])), DEPTH).scoreDocs;
          for (int i = 0; i < h.length; i++)
            w.printf(Locale.ROOT, "%s Q0 %s %d %.6f %s%n", q[0], sf.document(h[i].doc).get("id"), i + 1, h[i].score
     , e.getKey());
        }
      }
    }
    reader.close();
  }
}
```

ir_metrics.py

```
"""IR evaluation metrics shared by Part B and Part C. A ranked list is a list of doc IDs."""
import math

def precision(ranked, rel):
    return sum(d in rel for d in ranked) / len(ranked) if ranked else 0.0

def recall(ranked, rel):
    return sum(d in rel for d in ranked) / len(rel) if rel else 0.0

def f1(ranked, rel):
    p, r = precision(ranked, rel), recall(ranked, rel)
    return 2 * p * r / (p + r) if p + r else 0.0

def precision_at_k(ranked, rel, k):
    """The denominator stays k even if fewer than k documents were returned."""
    return sum(d in rel for d in ranked[:k]) / k

def reciprocal_rank(ranked, rel):
    for i, d in enumerate(ranked, 1):
        if d in rel:
            return 1 / i
    return 0.0

def average_precision(ranked, rel):
    """Relevant documents that were not retrieved contribute 0 (division by |rel|)."""
    hits, total = 0, 0.0
    for i, d in enumerate(ranked, 1):
        if d in rel:
            hits += 1
            total += hits / i
    return total / len(rel) if rel else 0.0

def dcg(gains):
    return sum(g / math.log2(i + 1) for i, g in enumerate(gains, 1))

def ndcg_at_k(ranked, grades, k):
```

```
    """grades: doc -> gain; unjudged documents have gain 0."""
    actual = dcg([grades.get(d, 0) for d in ranked[:k]])
    ideal = dcg(sorted(grades.values(), reverse=True)[:k])
    return actual / ideal if ideal else 0.0

def mean(xs):
    xs = list(xs)
    return sum(xs) / len(xs) if xs else 0.0
```

part_c_eval.py

```
"""Part C: evaluate saved runs with the Part B metric functions (unchanged) and test differences.
Usage: part_c_eval.py <data_dir> <results_dir>"""
import json, sys, collections
import numpy as np
from scipy.stats import wilcoxon
from ir_metrics import *

def load_qrels(path):
    R, G = collections.defaultdict(set), collections.defaultdict(dict)
    for l in open(path):
        q, _, d, r = l.split()
        if int(r) > 0:
            R[q].add(d); G[q][d] = 1     # this copy of the qrels is binary, so gain = 1
    return R, G

def load_run(path):
    run = collections.defaultdict(list)
    for l in open(path):
        p = l.split(); run[p[0]].append(p[2])
    return run

def evaluate(run, R, G):
    o = {}
    for q in R:
        r = run.get(q, [])
        o[q] = dict(P=precision(r, R[q]), R=recall(r, R[q]), F1=f1(r, R[q]),
                    P3=precision_at_k(r, R[q], 3), P10=precision_at_k(r, R[q], 10),
                    R10=recall(r[:10], R[q]), F1_10=f1(r[:10], R[q]),
                    AP=average_precision(r, R[q]), RR=reciprocal_rank(r, R[q]),
                    nDCG5=ndcg_at_k(r, G[q], 5), nDCG10=ndcg_at_k(r, G[q], 10))
    return o

if __name__ == "__main__":
    data, res = sys.argv[1], sys.argv[2]
    R, G = load_qrels(f"{data}/qrels.txt")
    models = ["TF-IDF", "BM25", "LM-Dirichlet"]
    variants = ["LMD-mu50", "LMD-mu100", "LMD-mu200", "LMD-mu500", "LMD-mu1000", "LMJM-l0.1", "LMJM-l0.3", "LMJM-l0
     .7", "BM25-b0.25", "BM25-b0.5", "BM25-b1.0"]
    per = {m: evaluate(load_run(f"{res}/partc_{m}.run"), R, G) for m in models + variants}
    agg = {m: {k: mean(x[k] for x in v.values()) for k in next(iter(v.values()))} for m, v in per.items()}
    qs = sorted(R, key=int)

    # What happens if query IDs are NOT renumbered to match the qrels
    rawmap = {l.split("\t")[0]: l.split("\t")[1] for l in open(f"{data}/queries.tsv")}
    raw = {rawmap[q]: r for q, r in load_run(f"{res}/partc_BM25.run").items()}
    wrong = evaluate(raw, R, G)
    wrong_ids = {k: mean(x[k] for x in wrong.values()) for k in ["AP", "P10", "RR"]}

    # Paired two-sided randomization test and Wilcoxon signed-rank test over queries
    rng = np.random.default_rng(618); tests = []
    for a, b in [("BM25", "TF-IDF"), ("BM25", "LM-Dirichlet"), ("TF-IDF", "LM-Dirichlet")]:
        for k in ["AP", "nDCG10", "RR", "P10"]:
            x = np.array([per[a][q][k] for q in qs]); y = np.array([per[b][q][k] for q in qs]); d = x - y
            signs = rng.choice([-1, 1], size=(100000, len(d)))
            pp = (np.abs((signs * d).mean(axis=1)) >= abs(d.mean()) - 1e-12).mean()
            tests.append(dict(a=a, b=b, metric=k, diff=float(d.mean()), wins=int((d > 0).sum()), losses=int((d < 0)
     .sum()),
                              ties=int((d == 0).sum()), p_perm=float(pp), p_wilcoxon=float(wilcoxon(x, y).pvalue)))

    # Query-level behaviour
    best = collections.Counter()
    for q in qs:
        v = {m: per[m][q]["AP"] for m in models}; top = max(v.values())
        ws = [m for m in models if abs(v[m] - top) < 1e-12]; best["tie" if len(ws) > 1 else ws[0]] += 1
    zero = {m: sum(per[m][q]["P10"] == 0 for q in qs) for m in models}
    spread = sorted(qs, key=lambda q: -(max(per[m][q]["AP"] for m in models) - min(per[m][q]["AP"] for m in models)
     ))[:5]
    text = {l.split("\t")[0]: l.split("\t")[2].strip() for l in open(f"{data}/queries.tsv")}
    examples = [dict(q=q, text=text[q], nrel=len(R[q]), **{m: per[m][q]["AP"] for m in models}) for q in spread]
    runs = {m: load_run(f"{res}/partc_{m}.run") for m in models}
    overlap = {f"{a}|{b}": mean(len(set(runs[a][q][:10]) & set(runs[b][q][:10])) / 10 for q in qs)
```

```
                for a, b in [("BM25", "TF-IDF"), ("BM25", "LM-Dirichlet"), ("TF-IDF", "LM-Dirichlet")]}
    ret = {m: mean(len(runs[m][q]) for q in qs) for m in models}

    json.dump(dict(agg=agg, wrong_ids=wrong_ids, tests=tests, best_AP=dict(best), zero_P10=zero, examples=examples,
      overlap10=overlap, retrieved_mean=ret),
              open(f"{res}/partc_results.json", "w"), indent=1)
    with open(f"{res}/partc_per_query.csv", "w") as f:
        keys = list(next(iter(per["BM25"].values())))
        f.write("model,query," + ",".join(keys) + "\n")
        for m in models:
            for q in qs: f.write(f"{m},{q}," + ",".join(f"{per[m][q][k]:.6f}" for k in keys) + "\n")
    for m in models + variants: print(f"{m:14}", {k: round(v, 4) for k, v in agg[m].items()})
    for k, v in [("wrong_ids", wrong_ids), ("best_AP", dict(best)), ("zero_P10", zero), ("overlap10", overlap), ("
     retrieved", ret)]: print(k, v)
    for t in tests: print(t)
    for e in examples: print(e)
```

verify.py

```
"""Independent check: recompute Part C metrics with the ranx library. Usage: verify.py <data_dir> <results_dir>"""
import json, sys
from ranx import Qrels, Run, evaluate as rx
from part_c_eval import load_qrels
data, res = sys.argv[1], sys.argv[2]
R, _ = load_qrels(f"{data}/qrels.txt")
Q = Qrels({q: {d: 1 for d in R[q]} for q in R})
own = json.load(open(f"{res}/partc_results.json"))["agg"]; out = {}
names = {"map": "AP", "mrr": "RR", "precision@3": "P3", "precision@10": "P10", "recall@10": "R10", "ndcg@5": "nDCG5
     ", "ndcg@10": "nDCG10", "recall@1000": "R"}
for m in ["TF-IDF", "BM25", "LM-Dirichlet"]:
    run = {}
    for l in open(f"{res}/partc_{m}.run"):
        q, _, d, rk, _s, _t = l.split(); run.setdefault(q, {})[d] = 1000.0 - int(rk)
    r = rx(Q, Run(run), list(names))
    out[m] = {k: (round(own[m][v], 6), round(float(r[k]), 6)) for k, v in names.items()}
    print(m, "all match:", all(abs(a - b) < 2e-6 for a, b in out[m].values()), out[m])
json.dump(out, open(f"{res}/verify_ranx.json", "w"), indent=1)
```